

pico-JEPA v2: ¿Cuándo Superan Muchos Modelos Pequeños a Uno Grande?

Adrián Rostagno, Javier Iparraguirre, Roberto Briatore, Agustín González, and
Santiago Aggio

Universidad Tecnológica Nacional, Facultad Regional Bahía Blanca, Bahía Blanca,
Argentina

{arostag, jiparraguirre, rbriatore, agonzalez, slaggio}@frbb.utn.edu.ar

Resumen pico-JEPA es un sistema que aprende a representar videos sin necesidad de etiquetas, pensado para correr en una computadora de escritorio. En esta segunda versión ponemos a prueba una idea sencilla: en lugar de entrenar un único modelo con todos los datos, entrenar varios modelos pequeños —cada uno con una porción distinta— y combinar sus predicciones. Evaluamos la idea sobre 30 clases de Kinetics-700, con una red compacta de unos 3.4 millones de parámetros entrenada en una sola placa gráfica de escritorio. El resultado principal: cuando los modelos del conjunto son diversos entre sí y sus predicciones se combinan con un meta-modelo aprendido (*apilamiento*), el conjunto supera al modelo único por +5.10 puntos porcentuales, con significancia estadística formal. Pero la ventaja no es universal: dos experimentos controlados muestran que desaparece si la red base es demasiado pequeña o si hay pocas clases. Más que una validación de sí o no, nuestra contribución central es un *mapa de condiciones* bajo las cuales la sabiduría colectiva de modelos pequeños supera al modelo único. El código se publica en <https://github.com/BHI-Research/pico-jepa-v2>.

Keywords: aprendizaje auto-supervisado de video, conjuntos de modelos, apilamiento, JEPA, ViT, Kinetics-700

1 Introducción

Enseñar a una computadora a entender videos suele exigir grandes cantidades de material etiquetado a mano, una tarea costosa y lenta. El aprendizaje auto-supervisado ofrece una alternativa: el modelo aprende mirando videos sin etiquetas. Dentro de esta familia, la arquitectura JEPA [1] aprende a predecir cómo se ve una región oculta del video a partir de la región visible, sin reconstruir cada pixel, y así obtiene representaciones más abstractas y útiles [2].

En este trabajo abordamos una pregunta complementaria: dado un presupuesto fijo de cómputo y datos, ¿conviene entrenar un único modelo con todo el material, o varios modelos pequeños, cada uno con una porción distinta, y luego combinar sus predicciones? Esta es la hipótesis central de *pico-JEPA* [19].

La intuición proviene del refrán «la unión hace la fuerza»: si cada modelo se equivoca de manera distinta, la decisión conjunta puede ser más acertada que la

de cualquiera por separado [12,13]. Sin embargo, esta ventaja no es automática: depende de la capacidad de cada modelo, de la calidad de lo aprendido y de cómo se combinen las predicciones [14]. Precisamente por eso, la pregunta interesante no es solo *si* el conjunto gana, sino *cuándo*.

Respecto de nuestro trabajo previo [19], que mostró la viabilidad de pico-JEPA en hardware de escritorio con votación simple sobre pocas clases, esta versión aporta cuatro elementos nuevos:

1. Un conjunto *heterogéneo*: cada modelo pequeño lleva una cabeza de clasificación distinta, para que aprendan cosas distintas.
2. El *apilamiento* como forma de combinar las predicciones: un meta-modelo que aprende a quién conviene escuchar en cada caso.
3. Un análisis estadístico formal, con intervalos de confianza y verificación de robustez.
4. Un *mapa de condiciones de validez*: dos experimentos controlados que delimitan cuándo la ventaja del conjunto aparece y cuándo desaparece.

El artículo desarrolla los conceptos a medida que los necesita y se apoya en tres diagramas que resumen, respectivamente, el funcionamiento del modelo (Fig. 1), la metodología de entrenamiento (Fig. 2) y la de evaluación (Fig. 3).

2 Trabajo Relacionado

Aprender de video sin etiquetas. Las técnicas auto-supervisadas para visión se agrupan en tres grandes familias. Los métodos *contrastivos* enseñan al modelo que dos vistas transformadas del mismo contenido deben tener representaciones parecidas [4]; BYOL mostró que la idea funciona incluso sin ejemplos negativos, apoyándose en un segundo codificador actualizado por promedio móvil [5], un mecanismo que la familia JEPA hereda. Los métodos de *reconstrucción enmascarada* ocultan partes de la entrada y las reconstruyen pixel a pixel: MAE lo demostró en imágenes [6] y VideoMAE en video [7], con notable eficiencia en datos. La familia JEPA [1], por último, también oculta regiones, pero las predice en el espacio de representaciones en lugar de reconstruirlas: I-JEPA [2] lo hizo en imágenes y V-JEPA [3] lo extendió al video, ocultando bloques espacio-temporales que abarcan varios cuadros a la vez. Esta última opción es especialmente atractiva en un escenario de bajo cómputo, porque evita el costo de un decodificador de píxeles y de reconstruir detalles irrelevantes para la semántica. Nuestro trabajo previo [19] adoptó la formulación de V-JEPA sobre un transformador de visión (ViT) [8] compacto, en hardware de escritorio; aquí la extendemos con un conjunto heterogéneo y un análisis estadístico formal.

Conjuntos de modelos. Combinar varias redes profundas mejora la exactitud, la confiabilidad y la robustez de las predicciones [12,13]. La diversidad entre los miembros —la condición para que el conjunto aporte— puede provenir de distintas inicializaciones [15], de darle a cada uno una porción distinta de los datos, o de variar su arquitectura o sus hiperparámetros [14]. Las formas de combinar también tienen su historia: frente a la votación y el promedio, la *generalización apilada* de Wolpert [9] propuso hace tres décadas que un segundo modelo *aprenda* de las predicciones de los primeros; es la base del apilamiento

que usamos aquí. En video, además, es habitual promediar la predicción sobre varios fragmentos del mismo clip para obtener evaluaciones más estables [16], práctica que adoptamos.

Especialización y reparto de datos. La idea de que cada modelo se especialice en una parte del problema se remonta a las mezclas de expertos [17], hoy renovadas a gran escala en arquitecturas dispersas donde una compuerta aprendida enruta cada entrada hacia unos pocos expertos [10]. A diferencia de aquellas, pico-JEPA reparte los datos de forma explícita y equilibrada entre submodelos, lo que hace la especialización transparente, fácil de controlar y sin costo de enrutamiento.

Experimentación automatizada. La búsqueda de configuraciones de este trabajo se apoya en la tradición del aprendizaje de máquina automatizado (AutoML) [11] y, más directamente, en herramientas recientes de prueba automatizada de hipótesis [20], donde un bucle propone experimentos, los ejecuta y registra sus resultados con la métrica bloqueada de antemano.

3 Cómo Funciona pico-JEPA v2

El sistema tiene dos etapas: primero aprende a *representar* videos sin usar etiquetas; después usa esas representaciones para *clasificar* acciones. Sobre esa base se construye el conjunto de modelos pequeños. La Fig. 1 resume el mecanismo de la primera etapa y la Fig. 2 el flujo completo de entrenamiento.

3.1 Primera Etapa: Aprender sin Etiquetas

La idea es un juego de «completar lo que falta». Cada video se divide en pequeños bloques espacio-temporales (parches). Al modelo se le muestra solo una parte de ellos (el *contexto*, un 20 %) y se le pide predecir el contenido del resto (el *objetivo*, un 80 % oculto). La clave de JEPA es que la predicción no ocurre pixel a pixel, sino en un espacio de representaciones abstractas: el modelo debe anticipar *qué* hay en la región oculta, no dibujarla.

Para lograrlo intervienen tres piezas: un **codificador en línea** que procesa los parches visibles, un **codificador objetivo** que produce las representaciones «correctas» de los parches ocultos, y un **predictor** que intenta acertarlas a partir del contexto. El codificador objetivo es una copia del codificador en línea que se actualiza lentamente (un promedio móvil de sus pesos) y nunca recibe gradiente; este detalle es esencial, porque evita que ambos codificadores colapsen a una representación trivial y constante. El codificador es una red ViT compacta de unos 3.4 millones de parámetros ($d=192$, profundidad 8, 8 cabezas de atención: aproximadamente la mitad del ViT-Small canónico de $d=384$).

Formalmente, la pérdida es el error cuadrático medio entre lo predicho y la representación objetivo, sobre el conjunto $\mathcal{T}$ de parches ocultos:

$$\mathcal{L} = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \|\hat{z}_i - \text{sg}[f_{\bar{\theta}}(\mathbf{v})_i]\|_2^2, \quad (1)$$

donde $\mathbf{v} \in \mathbb{R}^{C \times T \times H \times W}$ es el clip y $\text{sg}[z]$ indica que el codificador objetivo no recibe gradiente.

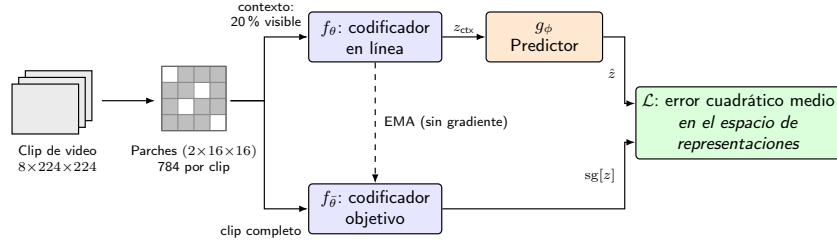


Figura 1. Funcionamiento del modelo (primera etapa, auto-supervisada).

3.2 Segunda Etapa: Clasificar

Una vez entrenado el codificador, le agregamos una *cabeza de clasificación*: un módulo pequeño que convierte las representaciones en una predicción de clase. Usamos tres tipos de cabeza, de menor a mayor complejidad: una **lineal** (capta señales generales promediadas), una **MLP de dos capas** (agrega matices no lineales) y una **atencional** (recupera mejor la estructura temporal del video). Esta variedad no es un detalle menor: como veremos, es la fuente de diversidad que hace funcionar al conjunto.

3.3 El Conjunto: Cuatro Modelos Pequeños que Deciden Juntos

El conjunto entrena $N=4$ modelos pequeños (*submodelos*). Cada uno recibe una porción distinta de los datos, repartida de forma equilibrada entre clases (cada submodelo ve una fracción $1/N$ de los ejemplos de cada clase), y se lo hace deliberadamente distinto de los demás mediante mecanismos de diversificación: rota el tipo de cabeza (lineal, MLP de dos capas, atencional), alterna si el codificador se congela o se ajusta finamente, varía la tasa de aprendizaje de la cabeza en $\pm 30\%$, usa una semilla distinta en cada uno ($\text{semilla_base} + 1000 \cdot i$) y varía la cantidad de épocas en ± 2 . El primer submodelo conserva la configuración base y actúa como referencia. El propósito es uno solo: que los submodelos *se equivoquen de maneras distintas*, porque solo entonces combinar sus opiniones aporta algo.

Comparamos cuatro maneras de combinar sus predicciones, de la más simple a la más elaborada:

1. **Votación dura:** gana la clase más votada.
2. **Votación blanda:** se promedian las probabilidades que asigna cada submodelo.
3. **Votación ponderada:** el promedio da más peso a los submodelos que mejor rindieron en validación.
4. **Apilamiento** [9]: un meta-modelo (una regresión logística) recibe las probabilidades de todos los submodelos y *aprende* a quién conviene escuchar para cada clase.

La diferencia conceptual es importante: las tres votaciones tratan a los submodelos con reglas fijas; el apilamiento, en cambio, aprende de los datos cómo ponderarlos. La Fig. 2 resume el flujo completo de entrenamiento, desde los videos sin etiquetas hasta la comparación entre el conjunto y el modelo único.

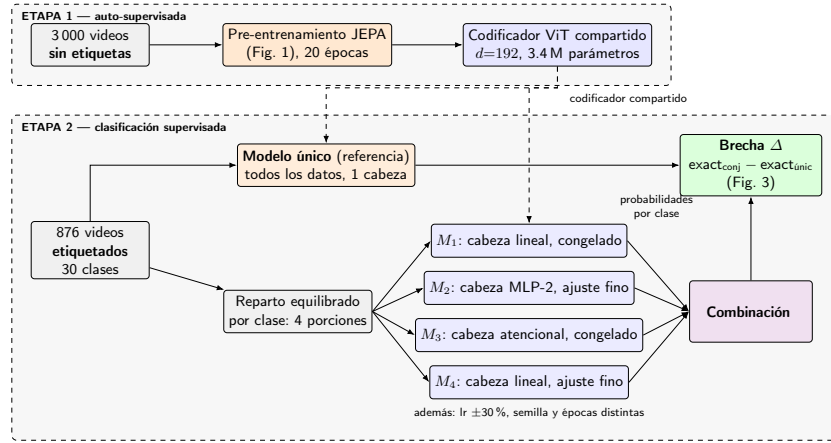


Figura 2. Metodología de entrenamiento. La Etapa 1 aprende un codificador sin etiquetas; la Etapa 2 lo reutiliza en dos ramas que se comparan entre sí: el *modelo único*, entrenado con todos los datos etiquetados, y el *conjunto* de cuatro submodelos, cada uno entrenado con una porción equilibrada por clase y con una cabeza distinta. Las probabilidades de los cuatro submodelos alimentan la combinación. La diferencia de exactitud entre ambas ramas es la brecha Δ , métrica única del trabajo.

4 Configuración Experimental

Datos. Usamos Kinetics-700-2020 (K700) [18], un conjunto público de reconocimiento de acciones humanas en video. El codificador se entrena sin etiquetas sobre 3000 videos; la clasificación usa 30 clases (1 467 videos), divididos en tres partes: entrenamiento (876), validación (297) y un *conjunto reservado* de 294 videos que no se toca hasta la evaluación final. Al evaluar, cada video se predice promediando sobre 10 fragmentos, una práctica habitual que estabiliza la medición [16]. La Fig. 3 resume el protocolo completo de evaluación.

La métrica: la brecha. Toda la evaluación gira en torno a una sola pregunta: ¿cuánto supera el conjunto al modelo único? Llamamos *brecha* (Δ) a esa diferencia de exactitud, en puntos porcentuales (pp), medida sobre el conjunto reservado. El modelo único —nuestra referencia— se entrena con todos los datos de entrenamiento; cada submodelo, solo con su porción.

¿La brecha es real o es azar? Con 294 videos de evaluación, una brecha positiva podría deberse a la suerte. Para descartarlo usamos *bootstrap*: remuestreamos el conjunto reservado 10 000 veces (semilla 1337) y observamos en qué rango cae la brecha, obteniendo un intervalo de confianza (IC) del 95%. El veredicto es **SOPORTADO** si todo el intervalo queda por encima de cero (el conjunto gana con 95% de confianza), **INCONCLUSO** si la brecha es positiva pero el intervalo contiene el cero, y **RECHAZADO** si la brecha no es positiva.

Hiperparámetros y hardware. El pre-entrenamiento usa 20 épocas, fracción oculta 0.80 con ocultamiento multibloque (3 bloques), promedio móvil $\tau \in [0.996, 0.9999]$ y decaimiento por capa 0.9. En la clasificación, el codificador puede mantenerse congelado o ajustarse finamente con tasa de aprendizaje reducida ($lr_{codificador} = 3 \times 10^{-5}$, $lr_{cabeza} = 5 \times 10^{-4}$). Todo el flujo corre en una

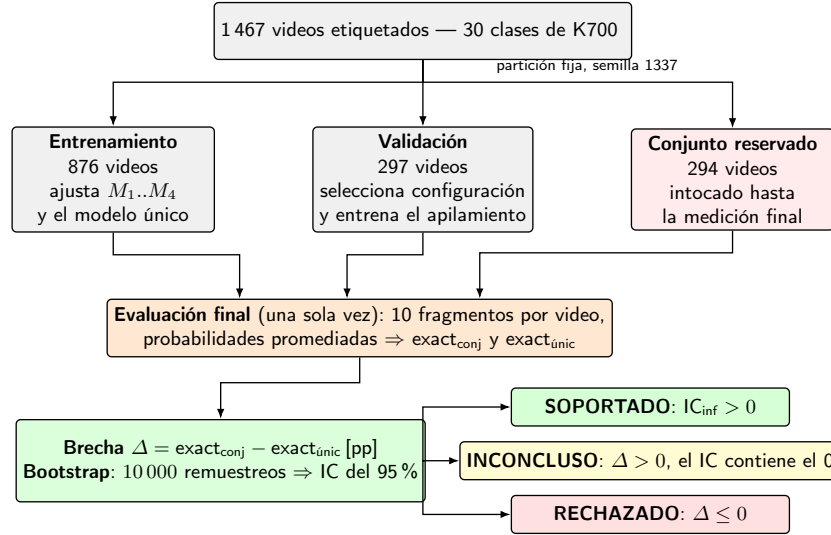


Figura 3. Metodología de evaluación. La triple partición se genera una sola vez con semilla 1337 y el conjunto reservado permanece inalterado durante toda la búsqueda de configuraciones: solo se accede a él en la medición final. La métrica oficial —la brecha Δ — y su intervalo de confianza por bootstrap determinan uno de los tres veredictos, fijados de antemano.

única máquina de escritorio (Intel Core i7, 65 GB de RAM, NVIDIA GTX 1060 de 6 GB).

5 Resultados

5.1 Una Señal Consistente: Nueve Mediciones, Nueve Positivas

La exploración de configuraciones no se hizo a mano: la condujo un bucle de experimentación automatizada, inspirado en *autoresearch* [20], que propone cambios de configuración, entrena los modelos reales y registra cada resultado, siempre con la métrica y el conjunto reservado fijados de antemano. Cada vez que una configuración mejoraba en validación, el bucle medía la brecha sobre el conjunto reservado. Ese proceso produjo nueve mediciones, y las nueve resultaron positivas, con un promedio de +2.53 pp (Fig. 4). Si la verdadera brecha fuera cero, obtener nueve positivos seguidos sería tan improbable como sacar nueve caras seguidas con una moneda ($0.5^9 \approx 0.002$).

Conviene ser precisos sobre qué prueba esto y qué no. Las nueve mediciones comparten el mismo conjunto reservado y provienen del mismo bucle, así que no son independientes entre sí: deben leerse como evidencia de *consistencia direccional* (todas las configuraciones favorecen al conjunto), no como una prueba formal. La prueba formal viene a continuación.

5.2 El Resultado Central: Solo el Apilamiento Gana con Significancia

En un experimento controlado, independiente del bucle de búsqueda, fijamos los cuatro submodelos y el modelo único, y variamos únicamente la forma de combinar las predicciones. La Tabla 1 y la Fig. 5 muestran el resultado.

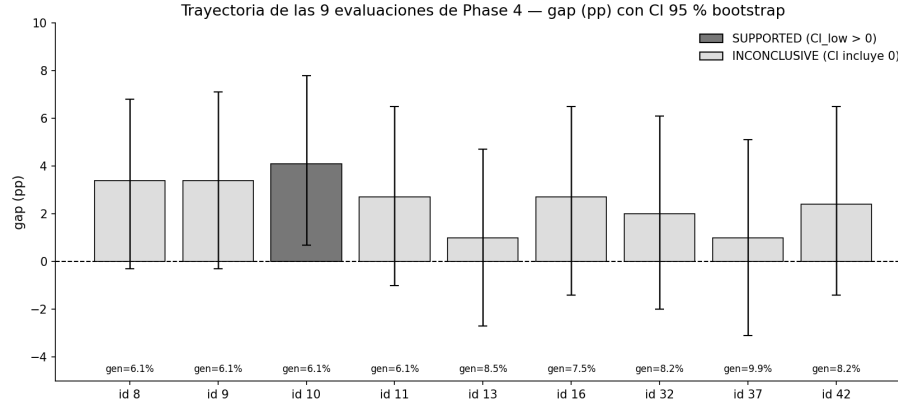


Figura 4. Las nueve mediciones de la brecha a lo largo del recorrido de búsqueda, con sus intervalos de confianza del 95 % (bootstrap). Todas son positivas; debajo de cada barra, la exactitud del modelo único en ese punto.

Tabla 1. Comparación de formas de combinar predicciones (4 submodelos heterogéneos; exactitud del modelo único: 6.80 %).

Combinación	Exactitud	Brecha	IC _{95%} inferior	Veredicto
Apilamiento	11.90 %	+5.10 pp	+0.68	SOPORTADO
Votación dura	8.84 %	+2.04 pp	-1.70	INCONCLUSO
Votación blanda	8.50 %	+1.70 pp	-2.04	INCONCLUSO
Votación ponderada	7.48 %	+0.68 pp	-2.72	INCONCLUSO

El aprendizaje es doble. Primero, el apilamiento es la única estrategia cuyo intervalo de confianza excluye el cero: el conjunto supera al modelo único por +5.10 pp con significancia formal, y el resultado se mantiene al repetir el análisis con cuatro semillas aleatorias distintas. Segundo —y quizás más revelador—, las tres votaciones simples no solo no alcanzan significancia: quedan *por debajo* del modelo único en exactitud absoluta. Promediar a ciegas las predicciones de submodelos que vieron pocos datos termina perjudicando el resultado, salvo que un meta-modelo aprenda a distinguir cuándo conviene escuchar a cada uno. La diversidad, por sí sola, no alcanza: hay que saber combinarla.

Dos precisiones estadísticas. Primero, como se evalúan cuatro formas de combinar, una corrección de Bonferroni ($\alpha = 0.0125$) endurecería el umbral de significancia, y el $IC_{\text{inf}} = +0.68$ pp del apilamiento es marginal bajo ese criterio más estricto; por eso la solidez del resultado no descansa en un único test, sino en la convergencia de tres líneas de evidencia: el bootstrap del apilamiento, las nueve mediciones direccionalmente consistentes y los dos experimentos controlados de la próxima sección. Segundo, al repetir el cálculo del intervalo con cuatro semillas independientes ($\{1337, 42, 7, 2024\}$) el IC_{inf} se mantuvo positivo en los cuatro casos (entre +0.68 y +1.02 pp, veredicto SOPORTADO en todos).

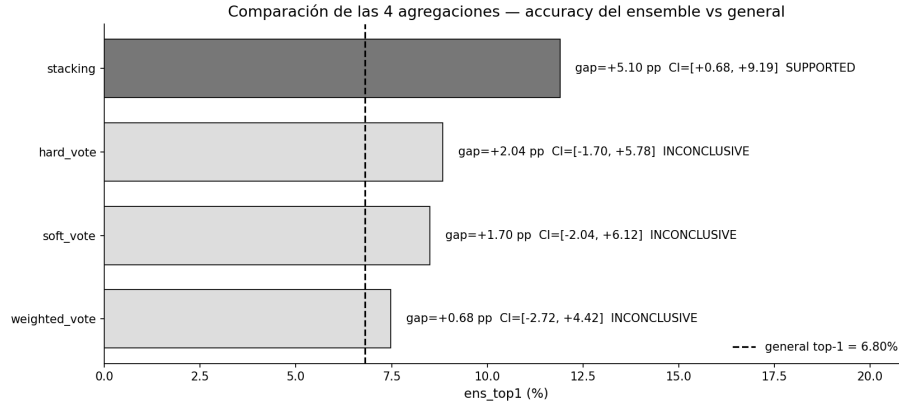


Figura 5. Exactitud del conjunto según la forma de combinar predicciones. La línea punteada marca la exactitud del modelo único (6.80 %). Solo el apilamiento (stacking) lo supera con significancia formal.

6 ¿Cuándo Funciona y Cuándo No?

La ventaja del conjunto no es universal. Dos experimentos controlados (Tabla 2) muestran sus límites, y son quizás el aprendizaje más útil de este trabajo.

Si la red base es demasiado pequeña, la ventaja desaparece. Repetimos todo el flujo con un codificador más chico (dimensión $d=128$ en lugar de 192). Aunque el modelo único mejora en términos absolutos (11.6 %), el apilamiento ya no aporta nada: la brecha cae exactamente a cero, con un intervalo de confianza degenerado ($[-0.04, +0.04]$). La interpretación es que, con menos capacidad, las distintas cabezas no logran extraer señales *diferentes* de las mismas características: los submodelos convergen a lo mismo, cometen los mismos errores, y no queda diversidad que combinar.

Si hay pocas clases, tampoco. Repetimos el flujo con 10 clases en lugar de 30. Concentrar los datos en menos clases hace al modelo único más fuerte (22.5 % de exactitud), pero el conjunto no logra superarlo: la brecha es incluso negativa (-1.5 pp). Con un espacio de clases estrecho, el reparto de datos le da a cada submodelo porciones muy parecidas, y de nuevo todos aprenden casi lo mismo.

Tabla 2. Experimentos controlados (apilamiento): la ventaja del conjunto requiere una red base con capacidad suficiente *y* un espacio de clases amplio.

Variante	d	Clases	Δ	Veredicto
Principal	192	30	+5.10 pp	SOPORTADO
Menor capacidad	128	30	+0.00 pp	RECHAZADO
Pocas clases	192	10	-1.50 pp	RECHAZADO

Los dos experimentos apuntan al mismo mecanismo: el conjunto vale lo que vale su diversidad. Cualquier factor que empuje a los submodelos hacia la misma solución —poca capacidad, pocas clases— anula la ventaja, por más que el apilamiento intente explotarla.

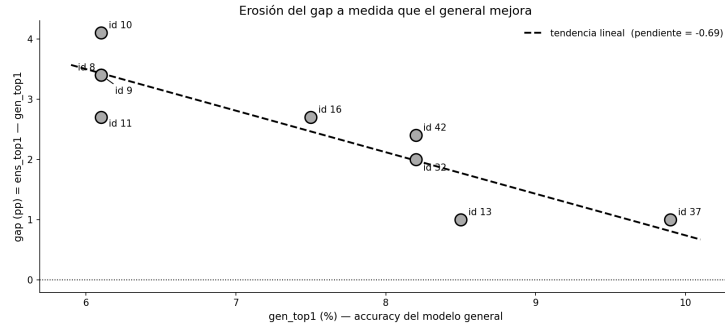


Figura 6. La brecha se achica a medida que el modelo único mejora. Cada punto es una medición del recorrido de búsqueda (pendiente ≈ -0.69 , $R^2 \approx 0.80$).

7 Análisis

Por qué el apilamiento es imprescindible. Cuando todos los submodelos tienen la misma cabeza, sus predicciones se parecen tanto que el meta-modelo no puede más que repetir el consenso, y la brecha se desvanece (así ocurría en una versión homogénea previa, con brecha ≈ 0). La diversidad de cabezas cambia esto: cada tipo de cabeza acierta y falla de manera distinta, y el apilamiento aprende a ponderarlas según convenga. Las votaciones simples, que tratan a todos por igual, no pueden hacerlo y quedan por debajo del modelo único.

La ventaja se achica cuando el modelo único mejora. La Fig. 6 relaciona la exactitud del modelo único con la brecha a lo largo de las nueve mediciones: a medida que el modelo único mejora, la brecha se reduce de forma marcada. Tiene sentido: el modelo único ve todos los datos y aprovecha mejor cada mejora del clasificador que un submodelo que solo ve una porción. La ventaja del conjunto es, entonces, más notoria cuando los modelos individuales son relativamente débiles, en línea con la literatura [13]. Al ser una relación observacional sobre nueve configuraciones, debe leerse como tendencia, no como efecto causal.

8 Limitaciones

Cuatro limitaciones acotan el alcance de las conclusiones. Primero, el conjunto reservado es chico (294 videos), por lo que los intervalos de cada medición individual son anchos. Segundo, las exactitudes absolutas son bajas (alrededor del 12 % sobre 30 clases, unas 3.6 veces el azar): reflejan que el codificador se entrenó con solo 3000 videos, unas 670 veces menos que el V-JEPA original. Tercero, la diversidad del conjunto se limita a la cabeza de clasificación: todos los submodelos comparten el mismo codificador, porque entrenar varios codificadores excedía la capacidad del hardware disponible. Cuarto, los resultados provienen de un único entrenamiento del codificador; la única incertidumbre cuantificada es la del bootstrap, y la variabilidad por inicialización queda fuera de los intervalos.

9 Conclusiones

La hipótesis central de pico-JEPA —un conjunto de modelos pequeños sobre porciones de datos supera a un único modelo entrenado con todos los datos—

queda validada con significancia estadística formal, pero solo cuando concurren tres condiciones: diversidad entre los submodelos, apilamiento como forma de combinar, y evaluación sobre múltiples fragmentos. La brecha es de +5.10 pp (IC 95 % [+0.68, +9.19], robusto ante cuatro semillas), y las nueve mediciones del recorrido de búsqueda son direccionalmente consistentes. Dos experimentos controlados marcan los límites: la ventaja se anula con una red base de menor capacidad o con pocas clases, porque en ambos casos los submodelos convergen a la misma solución y no generan la diversidad que el apilamiento explota.

El mensaje práctico para quien trabaje con recursos acotados: dividir los datos entre varios modelos pequeños puede ganar, pero no gratis. Hay que diseñar la diversidad (cabezas distintas), aprender la combinación (apilamiento en lugar de votación) y verificar que el problema tenga espacio suficiente (capacidad y clases) para que esa diversidad exista.

Como trabajo futuro proponemos ampliar el conjunto de entrenamiento sin etiquetas, lo que liberaría el techo de la clasificación, y ubicar el punto donde nace la ventaja caracterizando cómo cambia la brecha al variar la cantidad de clases entre 10 y 30 y la dimensión d de la red base.

Referencias

1. Y. LeCun, “A path towards autonomous machine intelligence”, *OpenReview*, 2022.
2. M. Assran et al., “Self-supervised learning from images with a joint-embedding predictive architecture”, in *CVPR*, 2023.
3. A. Bardes et al., “Revisiting feature prediction for learning visual representations from video”, *arXiv:2404.08471*, 2024.
4. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations”, in *ICML*, 2020.
5. J.-B. Grill et al., “Bootstrap your own latent: A new approach to self-supervised learning”, in *NeurIPS*, 2020.
6. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners”, in *CVPR*, 2022.
7. Z. Tong, Y. Song, J. Wang, and L. Wang, “VideoMAE: Masked autoencoders are data-efficient learners for self-supervised video pre-training”, in *NeurIPS*, 2022.
8. A. Dosovitskiy et al., “An image is worth 16×16 words: Transformers for image recognition at scale”, in *ICLR*, 2021.
9. D. H. Wolpert, “Stacked generalization”, *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
10. N. Shazeer et al., “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer”, in *ICLR*, 2017.
11. F. Hutter, L. Kotthoff, and J. Vanschoren, eds., *Automated Machine Learning: Methods, Systems, Challenges*, Springer, 2019.
12. T. G. Dietterich, “Ensemble methods in machine learning”, in *Multiple Classifier Systems*, Springer, 2000, pp. 1–15.
13. B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles”, in *NeurIPS*, 2017.
14. F. Wenzel et al., “Hyperparameter ensembles for robustness and uncertainty quantification”, in *NeurIPS*, 2020.
15. S. Fort, H. Hu, and B. Lakshminarayanan, “Deep ensembles: A loss landscape perspective”, *arXiv:1912.02757*, 2019.
16. L. Wang et al., “Temporal segment networks for action recognition in videos”, *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, pp. 2740–2755, 2018.
17. R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts”, *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
18. L. Smaira et al., “A short note on the Kinetics-700-2020 human action dataset”, *arXiv:2010.10864*, 2020.
19. A. Rostagno et al., “pico-JEPA: Comprendiendo el Video con Modelos Ultra-Ligeros y la Sabiduría Colectiva”, in *XXXI Congreso Argentino de Ciencias de la Computación (CACIC)*, 2025.
20. A. Karpathy, “autoresearch: automated hypothesis testing”, <https://github.com/karpathy/autoresearch>, 2025.